PROCESO DE DESARROLLO DE SOFTWARE

SOFTWARE VERDE.

MG. MOISÉS EVARISTO BUENO



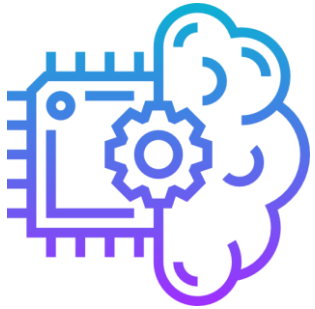
- QUE ES SOFTWARE VERDE



- Pilares Fundamentales



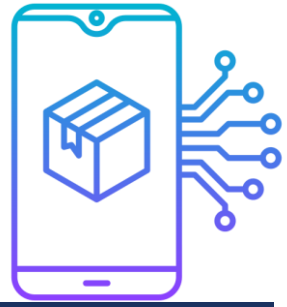
- Principios de Desarrollo



- Video Resumen general



- Consultas?



AGENDA DE TEMAS

SOFTWARE VERDE.

SOFTWARE VERDE (O *GREEN SOFTWARE*)



- El **software verde** (o *green software*) es una disciplina que busca diseñar, desarrollar y ejecutar aplicaciones informáticas minimizando su impacto ambiental. Su objetivo principal es optimizar el consumo de recursos para reducir las emisiones de carbono y la huella ecológica a lo largo de todo su ciclo de vida

PILARES FUNDAMENTALES



La industria del desarrollo sostenible se sostiene en tres ejes principales:

- **Eficiencia energética**: Escribir código optimizado que requiera menos electricidad para ejecutarse.
- **Conciencia del carbono**: Diseñar sistemas que adapten su consumo según la red eléctrica; por ejemplo, programar tareas pesadas para momentos del día donde la energía proviene de fuentes renovables y limpias.
- **Eficiencia del hardware:** Extender la vida útil de los equipos utilizando programas que no requieran cambiar constantemente a dispositivos más potentes

PRINCIPIOS DE DESARROLLO



Para lograr que un software sea considerado "verde", los equipos de ingeniería aplican prácticas conocidas como *Green Coding*, que incluyen:

- **Optimización de algoritmos**: Reducir la complejidad computacional para que los procesadores trabajen menos.
- **Gestión de datos**: Minimizar el almacenamiento innecesario en la nube y optimizar la transmisión de datos.
- **Arquitectura escalable**: Utilizar infraestructuras que se adapten a la demanda real, evitando servidores encendidos sin uso.

IMPACTO



1. Cómo medir el impacto (Herramientas)

No puedes optimizar lo que no mides. Existen herramientas específicas para calcular el consumo de carbono de tus aplicaciones:

- **Scaphandre**: Mide el consumo eléctrico (en vatios) de procesos específicos y servidores en entornos Linux.
- **Cloud Carbon Footprint**: Conéctala a AWS, Google Cloud o Azure para ver gráficos de las emisiones de carbono de tu infraestructura en la nube.
- **Green Frame**: Evalúa la huella de carbono de aplicaciones web simulando recorridos de usuarios comunes.
- **EcoCode**: Plugin para plataformas como SonarQube que escanea el código en busca de "malos hábitos" energéticos.

IMPACTO



2. Cómo mitigar el impacto (Estrategias)

Una vez identificados los puntos críticos, aplica las siguientes acciones de mitigación:

- **Optimización de Código e Infraestructura**
  - **Cambiar a servidores verdes**: Aloja tus sistemas en centros de datos que funcionen con energía 100% renovable.
  - **Usar arquitecturas *Serverless***: Los servicios basados en funciones (FaaS) solo consumen recursos cuando se ejecutan, evitando servidores encendidos en vacío.
  - **Implementar *Caching* agresivo**: Guarda en caché datos frecuentes para evitar consultas repetidas a la base de datos y transferencias de red innecesarias.
  - **Minificar y comprimir**: Reduce el tamaño de imágenes, scripts y archivos transferidos para disminuir el esfuerzo de las redes de comunicación.

IMPACTO



2. Cómo mitigar el impacto (Estrategias)

Una vez identificados los puntos críticos, aplica las siguientes acciones de mitigación:

- **Conciencia del Carbono (*Carbon Awareness*)**
  - **Planificación diferida**: Usa herramientas como la API de **Carbon Intensity** para programar tareas pesadas (como respaldos o entrenamiento de IA) en las horas del día donde la red eléctrica de tu región use más energía limpia (solar o eólica).

IMPACTO - LENGUAJES



Python: Optimizar la velocidad de ejecución

Python es un lenguaje interpretado y, por defecto, consume más energía que los lenguajes compilados. El objetivo aquí es reducir el tiempo que el procesador pasa ejecutando el código.

Usa funciones nativas y bibliotecas en C: Evita escribir bucles for manuales para operaciones matemáticas o de datos. Usa NumPy o Pandas, ya que sus operaciones internas están escritas en C y son altamente eficientes a nivel energético.

Reemplaza bucles por comprensión de listas: Las construcciones como [x for x in lista] se ejecutan a nivel de C dentro del intérprete de Python, siendo mucho más rápidas y eficientes que un bucle tradicional.

Implementa Generadores: Usa yield o expresiones generadoras en lugar de cargar listas masivas en la memoria RAM. Esto reduce drásticamente el consumo de memoria del servidor.

Evalúa cambiar de intérprete: Si tu aplicación procesa muchas tareas en paralelo, considera usar PyPy en lugar de CPython (el estándar). PyPy incluye un compilador JIT (Just-In-Time) que puede acelerar la ejecución hasta 4 veces, reduciendo el tiempo de uso del procesador

IMPACTO - LENGUAJES



JavaScript: Eficiencia en la Web y en el Servidor

En JavaScript, el impacto ocurre tanto en el navegador del usuario (consumo de batería) como en el servidor (Node.js).

Evita el Reflow y Repaint en el navegador: Manipular el DOM de manera desordenada obliga a la tarjeta gráfica y al procesador del usuario a trabajar de más. Agrupa los cambios del DOM o usa fragmentos (DocumentFragment).

Descarga de trabajo con Web Workers: Para scripts que requieran mucho cálculo en el navegador, usa Web Workers. Esto permite ejecutar código en hilos de fondo sin congelar la interfaz, optimizando el uso de los núcleos del procesador.

Controla los Memory Leaks en Node.js: Asegúrate de limpiar los temporizadores (clearTimeout, clearInterval) y eliminar referencias a objetos grandes que ya no uses. JavaScript usa un recolector de basura (Garbage Collector) que consume mucha energía si tiene que limpiar memorias saturadas constantemente.

Evita la sobre-arquitectura (Paquetes NPM): No importes librerías gigantescas si solo necesitas una función. Usa técnicas de Tree Shaking durante la compilación para eliminar el código muerto que el usuario no necesita descargar ni procesar.

IMPACTO - LENGUAJES



Java: Gestión de Memoria y Concurrencia

Java es un lenguaje robusto, pero sus aplicaciones (especialmente las empresariales) suelen tener una huella de memoria muy alta.

Sintoniza el recolector de basura (Garbage Collector): Un recolector de basura mal configurado puede pausar la aplicación constantemente (eventos Stop-the-World), disparando el uso del procesador. Configura recolectores modernos como G1GC o ZGC diseñados para ser más eficientes.

Usa tipos primitivos: Prefiere int, double o boolean en lugar de sus objetos equivalentes (Integer, Double, Boolean). Los objetos en Java añaden una sobrecarga de memoria considerable.

Optimiza la concatenación de texto: Nunca uses el operador + dentro de bucles para unir texto. Usa StringBuilder, ya que evita la creación constante de nuevos objetos innecesarios en la memoria Heap.

Considera la compilación nativa (GraalVM): Puedes compilar tus aplicaciones Java directamente a código nativo usando GraalVM. Esto elimina la necesidad de la Máquina Virtual de Java (JVM) en ejecución, logrando que la aplicación encienda en milisegundos y consuma hasta un 80% menos de memoria en el servidor

VIDEO RESUMEN



Ver video Desarrollo_de_Software.mp4



GRACIAS

MOBUENO@UADE.EDU.AR